

LINUX FOUNDATIONS

Linux Foundations

Operate the system. Trust the evidence.

Terminal · permissions · SSH · logs · systemd

6-hour workshop

50% theory + demos · 50% hands-on labs

Ubuntu 24.04 browser labs · macOS + Windows



By the end, you can recover your bearings

01 **Navigate**

Move through Linux, manipulate files, and use pipes without losing track of location.

02 **Control access**

Reason about users, groups, ownership, modes, sudo, and SSH keys.

03 **Diagnose**

Start and stop services, filter logs, identify a cause, and verify the repair.

Your browser is a window into another Linux system

BROWSER

**Safari, Edge, or Chrome
on macOS or Windows**

Shows instructions + terminal

`http://linux-lab.local`

SESSION

**Ubuntu 24.04
in an Incus container**

Runs `ls`, `sudo`, `apt`, `systemctl`

`learner@linux-lab:~$`



Browser outside. Linux inside.

Practice gets exactly half of the workshop

180 min

Theory ♦ live demos

- Linux, filesystems, terminal
- Identity, permissions, editor
- SSH, logs, systemd

180 min

Six participant labs

- Open an isolated lab
- Reset, solve, verify state
- Repair a failed service

Breaks can sit around the blocks; the instructional split stays 50 / 50.

Linux is the kernel; a usable system is a stack

Applications

ssh · apt · nano · your services

User space

shells · libraries · GNU tools · systemd

Linux kernel

processes · memory · networking · devices

Hardware / VM / container

CPU · RAM · disks · virtual devices

A distribution is a curated operating agreement

Same fundamentals

- Kernel interface
- Filesystem concepts
- Users and permissions
- OpenSSH + systemd

Different choices

- Package format
- Release cadence
- Default configuration
- Support lifecycle

Choose for context

- Ubuntu / Debian
- RHEL / Rocky / Alma
- Fedora
- Cloud/vendor images

Today: Ubuntu 24.04 LTS keeps every lab consistent.

Most investigations begin at five filesystem anchors

/

one root; every absolute path starts here

/etc

host-specific configuration

/home

human users' files

/var

changing data: logs, caches, queues

/usr

installed programs, libraries, shared data

Paths are coordinates; your working directory is the origin

ABSOLUTE

```
STARTSAT /  
  
/var/log/syslog  
/etc/ssh/sshd_config
```

Works from anywhere because it begins at the root.

RELATIVE

```
STARTSAT pwd  
  
../archive/report.txt  
./scripts/check.sh
```

Meaning changes when your working directory changes.

Safety check: pwd → inspect the target → change it → verify the result

The shell turns text into a process request

● ● ● PROMPT + COMMAND

```
ubuntu@linux-course:~/demo$ grep -i error app.log
```

grep

program

-i

option

error

argument

app.log

argument

Exit status 0 = success · non-zero = a condition or failure

Navigate by asking, moving, then confirming

Where am I?

`pwd`

What is here?

`ls -la`

Move

`cd /var/log`

Create

`mkdir -p notes/archive`

Inspect

`cat · less · head · tail`

Find

`find · grep`

Live demo: narrate state, not keystrokes

● ● ● BROWSER → SESSION → PIPELINE

```
whoami; hostname; pwd  
ls /
```

```
ls /; cd /var/log; pwd  
mkdir -p ~/demo/{in,out}  
printf 'ok\nerror: disk\n' > ~/demo/in/app.log
```

```
grep "error" ~/demo/in/app.log \  
| tee ~/demo/out/errors.txt
```

Predict → run → read output → verify state

Pipes connect small tools into an investigation

● ● ● ONE QUESTION, THREE FILTERS

```
journalctl -u ssh --since today | grep -i failed | tail -n 20
```

|

send output to the next command

>

replace a file with output

>>

append output to a file

Rule: inspect before ✦ — redirection happens before the command runs.

Linux authorizes identities, not job titles

USER

UID 1001
alice

PRIMARY ♦ SUPPLEMENTARY GROUPS

alice · courseops · sudo

IDENTITY CHECKS

```
whoami
id
groups
getent passwd alice
getent group courseops
```

Processes run as a user.
Files have an owner ♦ group.
Access checks combine both.

Use sudo for one deliberate action

Normal shell

least privilege
clear ownership
smaller blast radius

✦ sudo

One privileged command

● ● ● EXPLICIT INTENT

```
sudo apt install tree  
sudo systemctl restart ssh  
sudoedit /etc/ssh/sshd_config
```

Avoid living in a root shell. Ask: what exact command needs privilege?

Permissions are three triplets evaluated in order

- **rwX** **r-X** - - -

OWNER

GROUP

OTHERS

rwX

r-X

- - -

file: read · change · run directory: list names · create/remove · enter/traverse

4 ♦ 2 ♦ 1 makes numeric modes readable

r = 4

w = 2

x = 1

640 = rw- r-- ---

● ● ● SAME INTENT, VERIFIED

```
chmod 640 report.txt
chmod u=rw,g=r,o= report.txt
stat -c "%A %a %U:%G %n" report.txt
```


Shared work needs the right group, not world access

TEAM SHARE

```
sudo install -d \  
  -o root -g courseops -m 2770 \  
  /srv/course-share  
  
sudo chown alice:courseops status.txt  
sudo chmod 640 status.txt
```

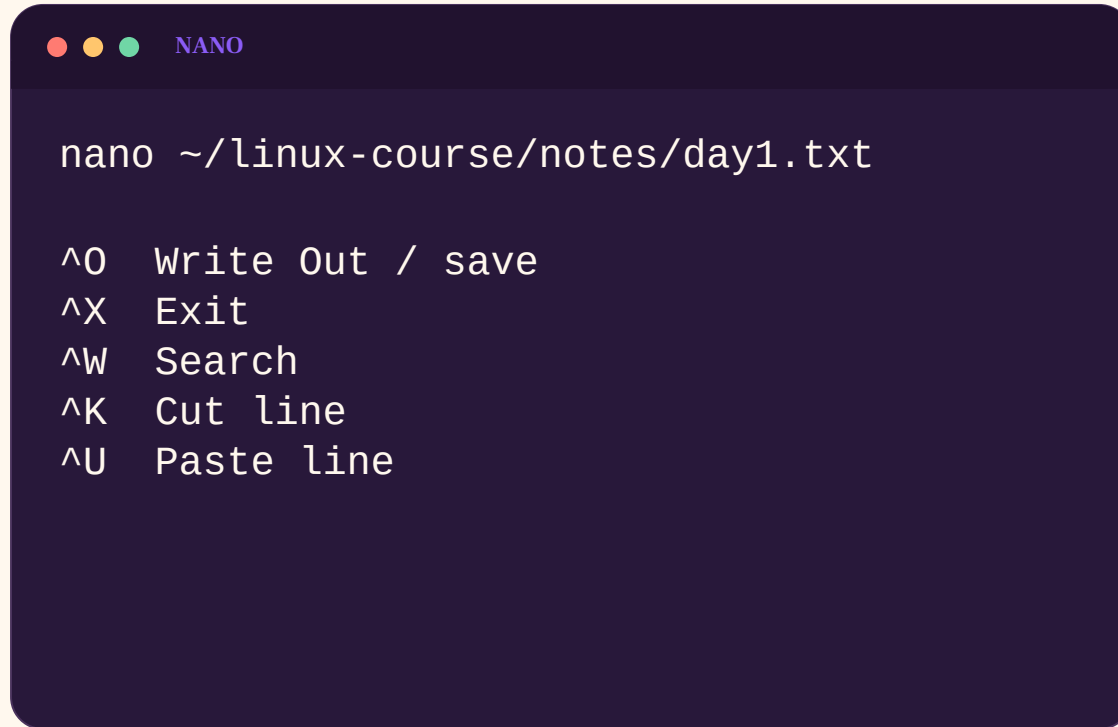
2

setgid on a directory

New files inherit the directory group, keeping collaboration predictable.

Owner ♦ group ♦ mode must tell the same story.

Nano is enough to make a safe first edit



```

NANO

nano ~/linux-course/notes/day1.txt

^O Write Out / save
^X Exit
^W Search
^K Cut line
^U Paste line
  
```

^ means Ctrl

- Open the exact path
- Make one small change
- Save, exit, then verify
- Use sudoedit for protected files

Editing is not verification.

Live demo: repair access with the smallest change

● ● ● OBSERVE → EXPLAIN → REPAIR → RETEST

```
id alice
stat -c "%A %a %U:%G %n" /srv/demo-share/report.txt
namei -l /srv/demo-share/report.txt

# observe failure as another identity
# repair owner/group/mode
# repeat the exact read
```

Never begin with chmod 777.

SSH authenticates both ends of the conversation

CLIENT

“Is this really my server?”

Checks the server host key
against `known_hosts`



SERVER

“Is this user authorized?”

Checks a signature against
`authorized_keys`

Encryption without host verification can still connect you to an attacker.

The private key proves; the public key permits

PRIVATE KEY

`~/.ssh/id_ed25519`

- Stays on the client
- Protect with a passphrase
- Never email or copy to server

PUBLIC KEY

`~/.ssh/id_ed25519.pub`

- May be copied to servers
- Stored in `authorized_keys`
- Can be revoked independently

● ● ● GENERATE ON THE CLIENT

```
ssh-keygen -t ed25519 -a 64 -C "linux-course"
```

Put safe SSH defaults in a named client profile

~/.ssh/config

```
Host linux-course
  HostName 192.168.64.10
  User trainee
  IdentityFile ~/.ssh/linux_course_ed25519
  IdentitiesOnly yes
  StrictHostKeyChecking accept-new
  ServerAliveInterval 60
```

- One alias replaces repeated flags
- IdentitiesOnly avoids key roulette
- Verify the first host fingerprint
- Use “yes” after onboarding for strictness

Same OpenSSH syntax on macOS and Windows.

Live demo: verify before you trust

● ● ● SERVER FINGERPRINT + USER KEY

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ''

ssh-keyscan -p 2222 localhost >> ~/.ssh/known_hosts
ssh-keygen -F '[localhost]:2222'
ssh-copy-id -i ~/.ssh/id_ed25519.pub -p 2222 operator@localhost

ssh -o BatchMode=yes -p 2222 \
  operator@localhost 'id; hostname'
```

Private key: client only. Public key: server. Host key: verify out of band.

Linux logs live in files and in the systemd journal

TEXT FILES

`/var/log`

- Application-specific files
- APT and dpkg history
- Rotation changes filenames

STRUCTURED JOURNAL

`journalctl`

- Unit, boot, PID, priority fields
- Time-aware filtering
- May be volatile or persistent

Start with the service and time window—not “all logs”.

Filter the journal toward one answer

Which service? `journalctl -u ssh`

Which time? `--since "-10 min"`

Which severity? `-p warning`

How much? `-n 50 --no-pager`

Live? `-f`

Live demo: ask the journal a smaller question

● ● ● UNIT → TIME → PRIORITY → FOLLOW

```
sudo systemctl restart ssh  
systemctl status ssh --no-pager
```

```
sudo journalctl -u ssh -n 10 --no-pager  
sudo journalctl -u ssh --since "-5 min" -p warning --no-pager  
sudo journalctl -u ssh -f
```

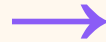
Read timestamps and the first causal message aloud.

systemd manages declared units toward a state

UNIT FILE

What should run?

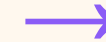
dependencies · user · command
restart policy · boot target



MANAGER

What is running?

start · stop · supervise
track exit status



JOURNAL

What happened?

events + output

“Enabled” means start at boot. “Active” means running now. They are independent.

Use systemctl verbs deliberately

Observe	<code>status</code> · <code>is-active</code> · <code>is-enabled</code>
---------	--

Change runtime	<code>start</code> · <code>stop</code> · <code>restart</code> · <code>reload</code>
----------------	---

Change boot	<code>enable</code> · <code>disable</code>
-------------	--

Inspect config	<code>cat</code> · <code>show</code>
----------------	--------------------------------------

After unit edits	<code>daemon-reload</code>
------------------	----------------------------

Restart a service only after capturing evidence.

Live demo: status is the symptom; logs reveal the cause

● ● ● FAILED SERVICE → VERIFIED ENDPOINT

```
systemctl status demo-web --no-pager
sudo journalctl -u demo-web --since "-5 min" --no-pager
systemctl cat demo-web

# repair missing user + working directory
sudo systemctl restart demo-web
systemctl is-active demo-web
curl -fsS http://localhost:8090
```

State ♦ logs ♦ configuration ♦ behavior

A safe operator closes every change with evidence

1

Locate

browser or session ·
user · directory

2

Predict

what state should
change?

3

Act

one scoped command

4

Observe

output · status · logs

5

Verify

behavior through
another check

Pair prompt: explain this loop using the failed-service demo.

HANDS-ON BLOCK

Three hours. One browser. Six checkpoints.

Work in pairs when useful. Driver types; navigator predicts and checks.

BROWSER LAB: open your assigned seat link

01

02

03

04

05

06

Labs 1–2 build the map, then turn data into evidence

LAB 1 • 25 MIN

Filesystem fieldwork

```
ORIENT • BUILD • VERIFY

p=~/projects/linux-course; pwd; whoami
mkdir -p "$p"/{notes,archive}
cp ~/inbox/welcome.txt "$p"/notes/
mv ~/inbox/draft.txt "$p"/archive/
```

LAB 2 • 25 MIN

Packages ♦ pipelines

- Install tree through APT
- Inspect package state
- Count HTTP status codes
- Write a verified report

Checkpoint: the project tree is correct and 404 appears exactly three times.

Labs 3–4 turn access rules into verified trust

LAB 3 · 35 MIN

Permission incident

● ● ● OWNER · GROUP · MODE

```
sudo chown -R root:webteam /srv/team-  
share  
sudo chmod 2770 /srv/team-share  
sudo -u alice test -w /srv/team-share  
sudo -u outsider test -w /srv/team-  
share || true
```

LAB 4 · 30 MIN

SSH trust setup

- Generate an Ed25519 key
- Record the server host key
- Authorize only the public key
- Connect through a safe alias

Checkpoint: Alice writes, outsider is blocked, and key-only SSH succeeds.

Labs 5–6 turn evidence into a durable repair

LAB 5 • 30 MIN

Journal investigation

- Start with service status
- Filter by unit + boot
- Find the first causal event
- Record incident + exit code

LAB 6 • 35 MIN

Broken service capstone

EVIDENCE → REPAIR

```
systemctl status course-web --no-pager
journalctl -u course-web -b --no-pager
systemctl cat course-web
sudo systemctl enable --now course-web
curl -fsS http://127.0.0.1:8088/
```

Final checkpoint: active ♦ enabled ♦ HTTP response.

When blocked, widen evidence before widening privilege

- 1 Context** `hostname · whoami · pwd`
- 2 Target** `ls -ld · stat · systemctl cat`
- 3 Symptom** `repeat the exact failing action`
- 4 Evidence** `status · journalctl · exit status`
- 5 Change** `one cause; then retest`

The final challenge is to explain, not merely succeed

Tell your partner:

- Which machine and identity ran the command?
- What evidence identified the service failure?
- Why did the permission repair preserve least privilege?
- How did you verify SSH server identity?
- What proved the web service behavior?

If you can explain the state, you can recreate the command.

Keep your seat link; keep practising the loop

NEXT 24 HOURS

- Reset Lab 6 and solve without hints
- Rebuild the SSH profile from memory
- Explain 640 and 2770 aloud

NEXT 30 DAYS

- Continue with Linux Upskill Challenge
- Read one man page per session
- Automate only after the manual path is clear

Locate → predict → act → observe → verify

Sources and open-course attribution

OPEN COURSE INSPIRATION

Linux Upskill Challenge · CC BY 4.0
linuxupskillchallenge.com

Linux Training + Linux101Labs
coverage and exercise-pattern review

PRIMARY REFERENCES

Canonical Ubuntu Server documentation
Incus system-container documentation
ttyd browser-terminal project
OpenSSH + systemd manuals

Visual direction

Platform source and delivery notes: [browser-lab-platform/README.md](#)

Full URLs, licensing notes, and adaptation details: [resources/SOURCES.md](#)